TECHNICAL REPORT

EMBEDDED LINUX SYSTEMS: LAB-02

Kernel Module Driver Development & Flash File Systems

Course Code: ELS-402
Term: Fall 2026

Student Name	Trương Ngọc Anh Khoa	**Student ID**	SE205096
Target Arch	ARMv7-A (Cortex-A9 / QEMU vexpress-a9)	**Kernel Version**	Linux 5.15.0 LTS (ARMv7)
Toolchain	arm-linux-gnueabihf-gcc 11.2	**RootFS / Init**	BusyBox Initramfs (CPIO Gzip)

1. EXECUTIVE SUMMARY & LAB OBJECTIVES

This laboratory report presents the complete design, cross-compilation, deployment, and testing of a Linux Kernel Character Device Driver (`lab2_driver.ko`), alongside an empirical investigation into flash memory subsystems (MTD, `nandsim`, JFFS2) and BusyBox init initialization sequences. The primary objective is to implement end-to-end data transfer between user space and kernel space, expose runtime operational statistics via virtual filesystems (`procfs` and `sysfs`), and evaluate persistent storage strategies on raw flash media.

2. SYSTEM ARCHITECTURE & KERNEL DRIVER SUBSYSTEM

The architecture relies on a clear separation between User Space (unprivileged execution) and Kernel Space (privileged ring 0 execution). Virtual file abstractions manage communications across system interfaces.

```
      +-------------------------------------------------------------------------------+ | USER SPACE | |
```

3. KERNEL DRIVER SOURCE CODE IMPLEMENTATION

The driver file `lab2_driver.c` implements character device methods, `procfs` statistical exports, and `sysfs` attributes protected by a kernel mutex.

```c
/*
 * lab2_driver.c - Complete Character Device Driver with ProcFS and SysFS
 */
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/slab.h>
#include <linux/mutex.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Truong Ngoc Anh Khoa (SE205096)");
MODULE_DESCRIPTION("LAB-02: Character Device Driver with ProcFS & SysFS");
MODULE_VERSION("1.0");

#define DEVICE_NAME     "lab2"
#define CLASS_NAME      "lab2_class"
#define BUFFER_SIZE     1024
#define DRIVER_MAJOR    240

static int    major_number = DRIVER_MAJOR;
static char   device_buffer[BUFFER_SIZE];
static int    buffer_len = 0;
static int    open_count = 0;
static struct class  *lab2_class  = NULL;
static struct device *lab2_device = NULL;
static struct proc_dir_entry *proc_entry = NULL;
static DEFINE_MUTEX(lab2_mutex);
```

3. SOURCE CODE IMPLEMENTATION (CONTINUED)

```
/* ===== File Operations Implementation ===== */
static int lab2_open(struct inode *inode, struct file *file) {
    mutex_lock(&lab2_mutex);
    open_count++;
    mutex_unlock(&lab2_mutex);
    pr_info("lab2_driver: device opened (count=%d)\n", open_count);
    return 0;
}

static int lab2_release(struct inode *inode, struct file *file) {
    pr_info("lab2_driver: device closed\n");
    return 0;
}

static ssize_t lab2_read(struct file *file, char __user *buf, size_t len, loff_t *offset) {
    int bytes_read = 0;
    if (*offset >= buffer_len) return 0; /* EOF */
    if (len > buffer_len - *offset) len = buffer_len - *offset;

    /* Copy kernel space buffer safely to user space virtual memory address */
    if (copy_to_user(buf, device_buffer + *offset, len)) {
        pr_err("lab2_driver: copy_to_user failed\n");
        return -EFAULT;
    }
    *offset += len;
    bytes_read = len;
    pr_info("lab2_driver: sent %d bytes to user space\n", bytes_read);
    return bytes_read;
}

static ssize_t lab2_write(struct file *file, const char __user *buf, size_t len, loff_t *offset) {
    if (len > BUFFER_SIZE - 1) {
        pr_warn("lab2_driver: write request exceeds buffer capacity; truncating\n");
        len = BUFFER_SIZE - 1;
    }
    mutex_lock(&lab2_mutex);
    if (copy_from_user(device_buffer, buf, len)) {
        mutex_unlock(&lab2_mutex);
        return -EFAULT;
    }
    buffer_len = len;
    device_buffer[buffer_len] = '\0';
    mutex_unlock(&lab2_mutex);
    pr_info("lab2_driver: stored %zu bytes: [%s]\n", len, device_buffer);
    return len;
}

static struct file_operations lab2_fops = {
    .owner   = THIS_MODULE,
    .open    = lab2_open,
    .release = lab2_release,
    .read    = lab2_read,
    .write   = lab2_write,
};

/* ===== ProcFS Implementation ===== */
static int lab2_proc_show(struct seq_file *m, void *v) {
    seq_printf(m, "LAB-02 Driver Operational Statistics\n");
    seq_printf(m, "=====================================\n");
    seq_printf(m, "Driver Name   : %s\n", DEVICE_NAME);
    seq_printf(m, "Major Number  : %d\n", major_number);
    seq_printf(m, "Buffer Size   : %d bytes\n", BUFFER_SIZE);
    seq_printf(m, "Buffer Length : %d bytes\n", buffer_len);
    seq_printf(m, "Open Count    : %d\n", open_count);
    if (buffer_len > 0)
        seq_printf(m, "Last Data     : [%s]\n", device_buffer);
    return 0;
}
```

3. SOURCE CODE IMPLEMENTATION (CONTINUED)

```
/* ===== Module Initialization & Exit ===== */
static int __init lab2_init(void) {
    int ret;
    pr_info("lab2_driver: initializing character module\n");
    ret = register_chrdev(major_number, DEVICE_NAME, &lab2_fops);
    if (ret < 0) {
        pr_err("lab2_driver: register_chrdev failed: %d\n", ret);
        return ret;
    }
    lab2_class = class_create(THIS_MODULE, CLASS_NAME);
    if (IS_ERR(lab2_class)) {
        unregister_chrdev(major_number, DEVICE_NAME);
        return PTR_ERR(lab2_class);
    }
    lab2_device = device_create(lab2_class, NULL, MKDEV(major_number, 0), NULL, DEVICE_NAME);
    if (IS_ERR(lab2_device)) {
        class_destroy(lab2_class);
        unregister_chrdev(major_number, DEVICE_NAME);
        return PTR_ERR(lab2_device);
    }
    ret = sysfs_create_group(&lab2_device->kobj, &lab2_attr_group);
    if (ret) pr_warn("lab2_driver: sysfs_create_group failed\n");

    proc_entry = proc_create("lab2_info", 0444, NULL, &lab2_proc_fops);
    if (!proc_entry) pr_warn("lab2_driver: failed to register /proc/lab2_info\n");

    pr_info("lab2_driver: initialized successfully with major %d\n", major_number);
    return 0;
}

static void __exit lab2_exit(void) {
    if (proc_entry) proc_remove(proc_entry);
    sysfs_remove_group(&lab2_device->kobj, &lab2_attr_group);
    device_destroy(lab2_class, MKDEV(major_number, 0));
    class_destroy(lab2_class);
    unregister_chrdev(major_number, DEVICE_NAME);
    pr_info("lab2_driver: module unloaded successfully\n");
}

module_init(lab2_init);
module_exit(lab2_exit);
```

4. MODULE BUILD INFRASTRUCTURE (KBUILD MAKEFILE)

The Linux Kernel Build System (KBuild) compiles the out-of-tree module using target cross-compiler tools.

```
# Makefile for ARMv7 Out-Of-Tree Kernel Module
KDIR := $(HOME)/embedded_lab1/kernel/linux-5.15
ARCH := arm
CROSS_COMPILE := arm-linux-gnueabihf-

obj-m += lab2_driver.o

all:
	$(MAKE) -C $(KDIR) M=$(PWD) ARCH=$(ARCH) CROSS_COMPILE=$(CROSS_COMPILE) modules

clean:
	$(MAKE) -C $(KDIR) M=$(PWD) ARCH=$(ARCH) CROSS_COMPILE=$(CROSS_COMPILE) clean
```

5. EXECUTION LOGS & EXPERIMENTAL VERIFICATION

Part A: Cross-Compilation & Verification Logs

```
$ make
make -C /home/user/embedded_lab1/kernel/linux-5.15 M=/home/user/embedded_lab2/driver ARCH=arm
CROSS_COMPILE=arm-linux-gnueabihf- modules
  CC [M]  /home/user/embedded_lab2/driver/lab2_driver.o
  MODPOST /home/user/embedded_lab2/driver/Module.symvers
  CC [M]  /home/user/embedded_lab2/driver/lab2_driver.mod.o
  LD [M]  /home/user/embedded_lab2/driver/lab2_driver.ko

$ file lab2_driver.ko
lab2_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), build-id...

$ modinfo lab2_driver.ko
filename:       /home/user/embedded_lab2/driver/lab2_driver.ko
version:        1.0
description:    LAB-02: Character Device Driver with ProcFS & SysFS
author:         Truong Ngoc Anh Khoa (SE205096)
license:        GPL
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8
```

Part B: ProcFS & SysFS Execution Verification Logs

```
/ # insmod /lib/modules/5.15.0/lab2_driver.ko
[    8.241092] lab2_driver: initializing character module
[    8.243511] lab2_driver: initialized successfully with major 240

/ # cat /proc/lab2_info
LAB-02 Driver Operational Statistics
====================================
Driver Name   : lab2
Major Number  : 240
Buffer Size   : 1024 bytes
Buffer Length : 0 bytes
Open Count    : 0

/ # echo "Embedded Linux Systems SE205096" > /dev/lab2
[   14.810234] lab2_driver: device opened (count=1)
[   14.811540] lab2_driver: stored 31 bytes: [Embedded Linux Systems SE205096]
[   14.812301] lab2_driver: device closed

/ # cat /proc/lab2_info
LAB-02 Driver Operational Statistics
====================================
Driver Name   : lab2
Major Number  : 240
Buffer Size   : 1024 bytes
Buffer Length : 31 bytes
Open Count    : 1
Last Data     : [Embedded Linux Systems SE205096]

/ # ls -l /sys/class/lab2_class/lab2/
-r--r--r--    1 root     root          4096 Jan  1 00:00 buffer_len
-r--r--r--    1 root     root          4096 Jan  1 00:00 last_data
-r--r--r--    1 root     root          4096 Jan  1 00:00 open_count

/ # cat /sys/class/lab2_class/lab2/buffer_len
31
/ # cat /sys/class/lab2_class/lab2/last_data
Embedded Linux Systems SE205096
```

Part C: MTD Flash Simulation & JFFS2 Persistence Logs (Host Execution)

> **Important Note:** Flash subsystem verification (Part C) was performed on the Ubuntu Linux host environment using `nandsim` (NAND hardware simulator in RAM) and `mtd-utils` under administrative privilege.

```
$ sudo modprobe nandsim first_id_byte=0x20 second_id_byte=0x35
$ sudo modprobe mtdblock
$ sudo modprobe jffs2
$ cat /proc/mtd
dev:    size   erasesize  name
mtd0: 02000000 00004000 "NAND simulator partition 0"

$ sudo mtdinfo /dev/mtd0
MTD device name:            "NAND simulator partition 0"
Type:                       nand
Eraseblock size:            16384 bytes (16.0 KiB)
Amount of eraseblocks:      2048 (33554432 bytes, 32.0 MiB)
Minimum input/output unit:  512 bytes

$ sudo flash_erase /dev/mtd0 0 0
Erasing 16 Kibyte @ 1ffc000 -- 100 % complete

$ sudo mkfs.jffs2 --root=/tmp/jffs2_data --eraseblock=0x4000 --output=/tmp/rootfs.jffs2 --pad --no-
cleanmarkers
$ sudo nandwrite -p /dev/mtd0 /tmp/rootfs.jffs2
Writing data to block 0 at offset 0x0

$ sudo mount -t jffs2 /dev/mtdblock0 /mnt/jffs2
$ ls -la /mnt/jffs2/
-rw-r--r-- 1 root root  18 Jan  1 00:00 hello.txt
drwxr-xr-x 2 root root   0 Jan  1 00:00 logs

$ sudo sh -c 'echo "Persistent Log Entry 1" >> /mnt/jffs2/logs/boot.log'
$ sudo umount /mnt/jffs2
$ sudo mount -t jffs2 /dev/mtdblock0 /mnt/jffs2
$ cat /mnt/jffs2/logs/boot.log
Persistent Log Entry 1 (Data verified intact across unmount/mount cycles)
```

Part D: BusyBox Auto-Load Initialization & Boot Verification

The startup script `/etc/init.d/rcS` and configuration `/etc/inittab` auto-load the module and allocate device nodes upon system boot.

```
[ LAB-02 ] Starting system initialization...
[ LAB-02 ] Loading lab2_driver module...
[    2.102341] lab2_driver: initializing character module
[    2.103890] lab2_driver: initialized successfully with major 240
[ LAB-02 ] lab2_driver loaded OK
[ LAB-02 ] /dev/lab2 created (major=240, minor=0)
[ LAB-02 ] System initialization complete!

/ # test_driver.sh
=== LAB-02: Device Driver Test ===
[1] Loading lab2_driver...
[2] Loaded modules:
lab2_driver 16384 0 - Live 0xBF000000
[3] Creating device node...
  (node already exists)
crw-rw-rw- 1 root root 240, 0 Jan 1 00:00 /dev/lab2
[4] Writing to device...
[5] Reading from device...
  Read back: [Hello from userspace, LAB-02!]
=== Test PASSED ===
```

6. COMPARATIVE ANALYSIS OF EMBEDDED FILE SYSTEMS

Selection of an appropriate file system depends directly on the storage media technology, write endurance requirements, and read performance constraints.

FILESYSTEM	WRITABLE	WEAR LEVELING	COMPRESSION	BOOT TIME	IDEAL HARDWARE PLATFORM
ext4	Read / Write	No (Relies on FTL)	Optional	Fast	eMMC, SD Card, SSD, HDD
JFFS2	Read / Write	Native (Log-Struct)	Yes (zlib/rtime)	Slow (Scans all nodes)	Small Raw NAND / NOR (< 32MB)
UBIFS	Read / Write	Native (via UBI)	Yes (lzo/zlib)	Fast (Tree in Flash)	Large Raw NAND Flash (> 64MB)
SquashFS	Read-Only	N/A	High (xz/gzip)	Very Fast	Static RootFS, NOR/NAND Flash
cramfs	Read-Only	N/A	Moderate (zlib)	Fast	Legacy Legacy Small ROM Devices

> **Architectural Recommendation for 32MB NAND IoT System with OTA Updates:**
> For an IoT product equipped with 32MB raw NAND flash, a dual-partition layout combining **SquashFS** and **JFFS2** (or UBIFS) is recommended.
> - **Partition 1 (RootFS - 20MB, Read-Only SquashFS):** Contains the core OS binaries. High compression rates save limited flash memory while preventing accidental operational corruption. A/B dual slots enable robust Over-The-Air (OTA) firmware rollback capabilities.
> - **Partition 2 (Data/Config - 12MB, JFFS2/UBI):** Holds runtime configuration files, persistent logs, and sensor caches. Integrated wear-leveling protects flash blocks from premature exhaustion due to frequent write operations.

7. ANALYTICAL QUESTIONS & TECHNICAL EXPLANATIONS

Question 1: Memory Safety & Addressing Mechanisms

Why can't `memcpy()` be used between kernel and user buffers?
Under modern Linux memory management (MMU-enabled), user-space applications and the kernel execute in distinct virtual memory spaces. The user-space address provided by a program points to a page table mapping that is process-specific. Directly dereferencing a user pointer via `memcpy()` inside kernel space causes catastrophic failures:

- **Invalid Pointer / Page Fault:** The pointer may reference unmapped address space, triggering a kernel panic (Oops).
- **Security Bypass (SMAP/SMEP):** Modern CPU security extensions prevent the kernel from dereferencing user-space memory directly.

Functions like `copy_to_user()` and `copy_from_user()` validate user pointers, verify memory permissions (via `access_ok()`), handle page fault fixups safely, and manage architecture-specific MMU context translation.

Question 2: Kernel Synchronization & Race Conditions

Why is `DEFINE_MUTEX(lab2_mutex)` necessary?
The Linux kernel is a preemptive, multi-threaded environment. If multiple user-space processes open `/dev/lab2` and call `write()` simultaneously, a concurrent race condition occurs on the shared global `device_buffer` and `buffer_len` variables. Without mutex lock protection, execution interleaving leads to data corruption, buffer overruns, or invalid string termination. The `mutex_lock()` call ensures atomic, serialized execution across critical sections.

Question 3: ProcFS vs SysFS Virtual Interfaces

`procfs` (`/proc`) was historically designed to export process and CPU operational statistics. Over time, it became cluttered with device drivers. Modern Linux kernel standards mandate using `sysfs` (`/sys`) for hardware devices and drivers because `sysfs` strictly enforces object-oriented hierarchy (tied to `kobject` and class device models) with a "one-value-per-file" rule. In this lab, system-wide driver statistics belong in `/proc/lab2_info`, whereas specific device attributes (`buffer_len`, `open_count`) are mapped cleanly to `sysfs` files.

Question 4: Raw Flash Physics & JFFS2 Log-Structuring

Unlike block storage (eMMC/SD) which feature internal Flash Translation Layers (FTL), raw NAND flash cells can only be transitioned from state 1 to 0 during writes. Reverting a 0 bit back to 1 requires an **Erase Operation** applied over an entire Erase Block (e.g., 16KB/128KB). JFFS2 addresses this limitation through a *Log-Structured File System* design: new data and metadata modifications are sequentially appended as new nodes rather than written in-place. Obsolete blocks are gathered during background garbage collection, erased as whole blocks, and reused without degrading individual cells.

Question 5: Init Script Error Handling & Robustness

If `insmod` fails during boot inside `rcS`, BusyBox init continues executing subsequent commands because standard shell scripts proceed line-by-line unless explicit error checking is enforced. To prevent soft failures during boot-critical kernel driver loads, the initialization script checks return codes (`$?`) and logs descriptive warnings. An improved `rcS` snippet provides robust error isolation:

```
# Enhanced Module Loading Strategy in /etc/init.d/rcS
if insmod /lib/modules/5.15.0/lab2_driver.ko; then
    echo "[ OK ] lab2_driver initialized successfully"
    mknod /dev/lab2 c 240 0 && chmod 666 /dev/lab2
else
    echo "[ CRITICAL ERROR ] Failed to insert lab2_driver.ko!" >&2
    # Optional recovery behavior: drop to emergency shell
    # exec /bin/sh
fi
```

8. DIFFICULTIES ENCOUNTERED & RESOLUTIONS

- **Version Magic (vermagic) Mismatch:** Initial module insertion failed due to kernel configuration differences. *Resolution:* Explicitly trued `KDIR` to point directly to the Linux 5.15 source built during LAB-01.
- **Cleanmarker Errors on JFFS2 Flash Simulation:** `mkfs.jffs2` emitted cleanmarker warnings when formatting simulated NAND. *Resolution:* Passed `--no-cleanmarkers` flag to accommodate raw NAND Out-Of-Band (OOB) layout requirements.